Department of Computer Engineering COMSATS University Islamabad, Attock Campus

Lab Rubrics Form		
Lab Name	Discrete-Time Signal Convolution	
Lab Number	**9**	
Submitted by:		
Student Names		**Registration #**
Sumaira Safeer		FA22-BCE-019

Rubrics		**Marks**		
		1	**2**	**3**
PLO-5 Modern Tools Usage	Ability to execute the experiment utilizing hardware, software tools, or a combination of both.			
PLO-10 Communication Skills	Present lab activities effectively through detailed lab reports, oral examinations, and engaging presentations.			

Lab-09

Discrete-Time Signal Convolution

Objective:

The objective of this lab was to understand and implement discrete-time convolution using multiple methods. Specifically, the tasks included:

1. Performing convolution between two given discrete-time signals using:
   - MATLAB's built-in conv() function.
   - Manual implementation using nested loops.
   - Representation as a sum of shifted and scaled impulse responses.
2. Visualizing the input signals, impulse response, and convolution output through stem plots.

Lab Task:

Compute convolution of following signals by using 'conv' command and loops.

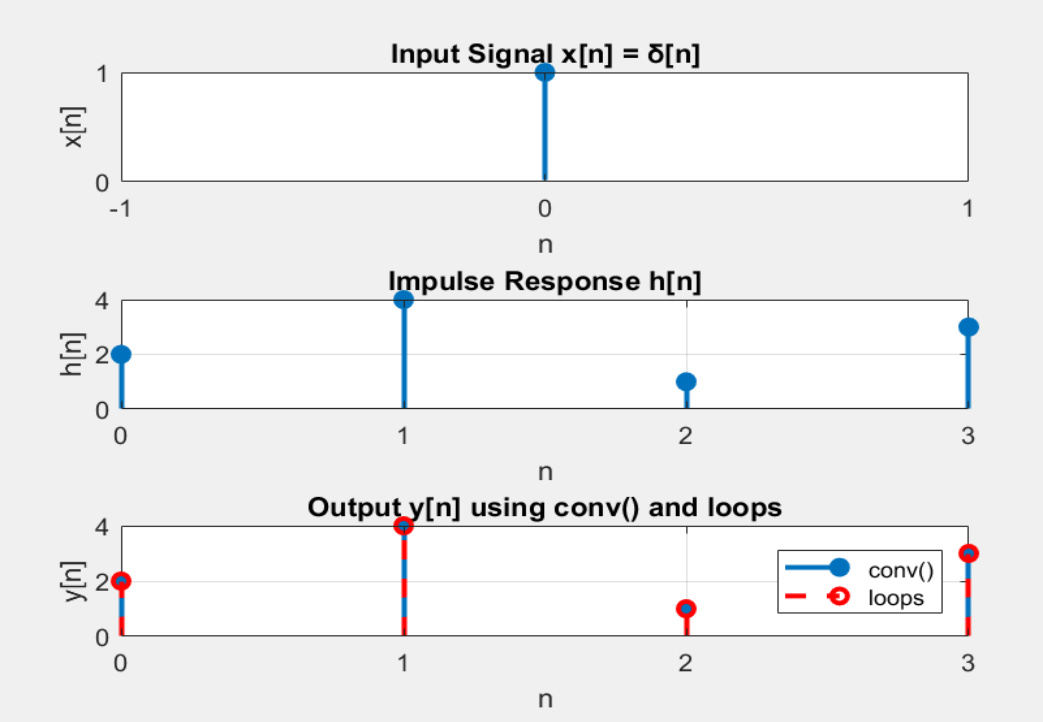
$$x[n]=\delta[n]=\begin{cases}1 & n=0\\ 0 & n\neq 0\end{cases} \qquad h[n]=[2\ 4\ 1\ 3], 0\le n\le 3$$

Code:

```
clc; clear; close all;
x = [1];              % delta signal ?[n]
h = [2 4 1 3];        % impulse response
y_conv = conv(x, h);
disp('Convolution using conv():');
disp(y_conv);
% (B) Convolution using loops (manual method)
Lx = length(x);
Lh = length(h);
Ly = Lx + Lh - 1;
y_loop = zeros(1, Ly);
for n = 1:Ly
   temp_sum = 0;
   for k = 1:Lx
      if (n-k+1 > 0) && (n-k+1 <= Lh)
         temp_sum = temp_sum + x(k) * h(n-k+1);
      end
```

```
    end
  y_loop(n) = temp_sum;
end
disp('Convolution using loops:');
disp(y_loop);
xticks(n);
% (C) Plotting results
n_x = 0:length(x)-1;
n_h = 0:length(h)-1;
n_y = 0:length(y_conv)-1;
figure;
subplot(3,1,1)
stem(n_x, x, 'filled');
title('Input Signal x[n] = ?[n]');
xlabel('n'); ylabel('x[n]'); grid on;
xticks(-1:1);
yticks(0: 2);
subplot(3,1,2)
stem(n_h, h, 'filled');
title('Impulse Response h[n]');
xlabel('n'); ylabel('h[n]'); grid on;
xticks(0:3);
subplot(3,1,3)
stem(n_y, y_conv, 'filled'); hold on;
stem(n_y, y_loop, 'r--');
title('Output y[n] using conv() and loops');
legend('conv()', 'loops');
xlabel('n'); ylabel('y[n]'); grid on;
xticks(0:3);
```

Output:



Lab Task 02:

Compute convolution of following signals by using 'conv' command and loops and as sum of shifted and scaled impulse response.

$$x[n]=\begin{cases}1 & 0\le n\le 4\\ 0 & elsewhere\end{cases}$$

and

$$h[n]=\begin{cases}1.5^n & 0\le n\le 6\\ 0 & elsewhere\end{cases}$$

Code:

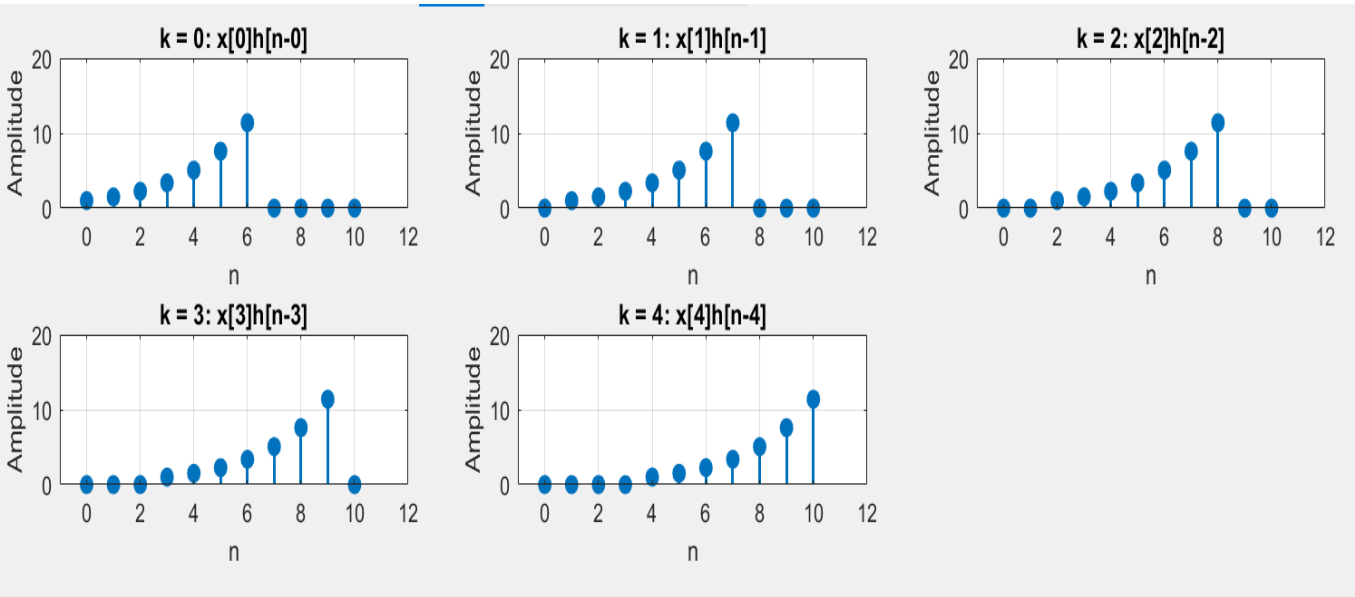
```
clc; clear; close all;
% Define signals
x = ones(1,5);        n_x = 0:4;        % x[n] = 1, n=0..4
h = 1.5.^(0:6);       n_h = 0:6;        % h[n] = 1.5^n
Ly = length(x)+length(h)-1;
% Convolution using built-in conv()
```

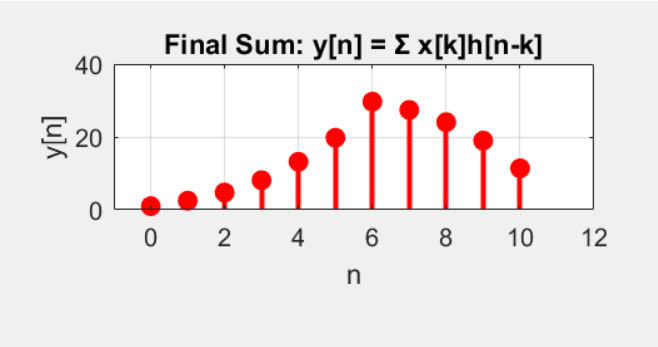
```
y_conv = conv(x,h);
n_conv = 0:Ly-1;
% Convolution using loops
y_loop = zeros(1,Ly);
for n = 0:Ly-1
   for k = 0:length(x)-1
      if n-k >=0 && n-k < length(h)
         y_loop(n+1) = y_loop(n+1) + x(k+1)*h(n-k+1);
      end
   end
end
% Sum of shifted/scaled h[n]
y_shifted = zeros(1,Ly);
figure('Position',[100,100,1200,800]);
for k = 0:length(x)-1
   h_shifted = zeros(1,Ly);
   h_shifted(k+1:k+length(h)) = x(k+1)*h;
   y_shifted = y_shifted + h_shifted;
   subplot(4,3,min(k+1,12));
   stem(0:Ly-1,h_shifted,'filled','LineWidth',1.5);
   title(sprintf('k=%d: x[%d]h[n-%d]',k,k,k));
   xlabel('n'); ylabel('Amplitude'); grid on; xlim([-1,Ly+1]);
   yticks(0:5:15);   % Set y-axis ticks to 0,5,10,15
end
subplot(4,3,12);
stem(0:Ly-1,y_shifted,'r','filled','LineWidth',2);
title('Final Sum y[n] = ? x[k]h[n-k]');
xlabel('n'); ylabel('y[n]'); grid on; xlim([-1,Ly+1]);
yticks(0:5:15);  % Set y-axis ticks
% Verify all methods
if max(abs(y_conv - y_loop))<1e-10 && max(abs(y_conv - y_shifted))<1e-10
   disp('? All methods match.');
else
   disp('? Methods differ.');
end
```

```
% Plot signals
figure('Position',[100,100,1200,600]);
subplot(2,3,1);
stem(n_x,x,'filled'); title('x[n]'); xlabel('n'); grid on;
yticks(0:5:15);
subplot(2,3,2);
stem(n_h,h,'filled','Color',[0.8 0.2 0.2]); title('h[n]'); xlabel('n'); grid on;
yticks(0:5:15);
subplot(2,3,3);
stem(n_conv,y_conv,'filled','Color',[0 0.5 0]); title('y[n] conv()'); xlabel('n'); grid on;
yticks(0:5:15);
subplot(2,3,[4 5 6]);
hold on;
stem(n_conv,y_conv,'filled','DisplayName','conv()');
stem(n_conv,y_loop,'ro','DisplayName','loops');
stem(n_conv,y_shifted,'k^','DisplayName','shifted');
title('Comparison of Methods'); xlabel('n'); ylabel('y[n]'); legend; grid on; hold off;
yticks(0:5:15);  % Set y-axis ticks
% Display results
disp('n   conv()   loops   shifted');
for n=0:Ly-1
   fprintf('%2d  %7.4f  %7.4f  %7.4f\n', n, y_conv(n+1), y_loop(n+1), y_shifted(n+1));
end
```

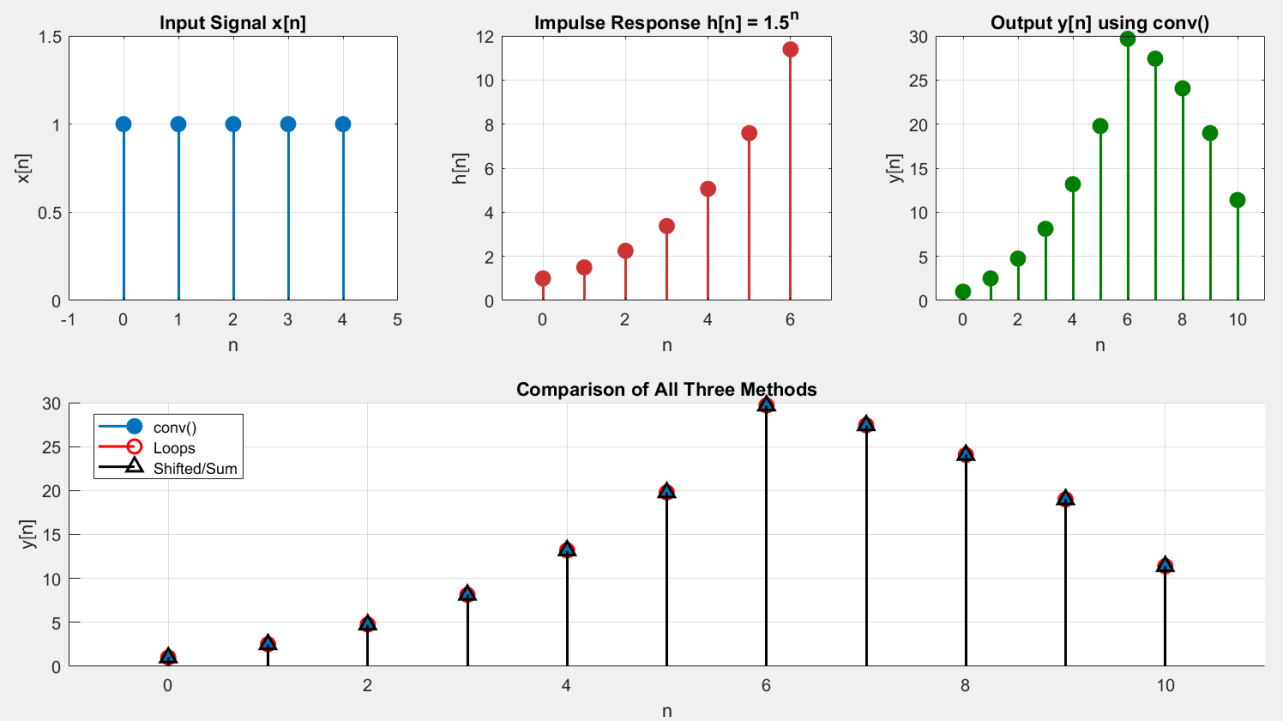
Output:

Computing Convolution on each index:


k = 0: x[0]h[n-0]
k = 1: x[1]h[n-1]
k = 2: x[2]h[n-2]
k = 3: x[3]h[n-3]
k = 4: x[4]h[n-4]
Amplitude
n


Final Sum: y[n] = Σ x[k]h[n-k]
y[n]
n

Result:


Input Signal x[n]
x[n]
n
Impulse Response h[n] = 1.5^n
h[n]
n
Output y[n] using conv()
y[n]
n
Comparison of All Three Methods
conv()
Loops
Shifted/Sum
y[n]
n